DS1707

NEURO-FUZZY COMPUTING

LABORATORY

DS1707	NEURO-FUZZY COMPUTING LABORATORY	L	T	P	C
		0	0	4	2

OBJECTIVES

- Understand Fuzzy concepts
- Learn neural networks with back propagation and without preparation
- Learn the operators of genetic algorithms
- Practice on crisp partitions.

LIST OF EXPERIMENTS

1. Implementation of Perceptron.	CO1
2. Implementation of Perceptron Rule	
3. Implementation of Artificial Neural Networks	
4. Implementation of Fuzzy Sets	CO2
5. Implementation of Covariance	
6. Data Fitting by Regression	
7. Implementation of Crisp Model	
8. Implementation of Logic Gates	CO3
9. Implementation of Genetic Algorithms	
10. Implementation of Classification Algorithm	

TOTAL : 60 PERIODS

REFERENCE BOOK

1. D.K Prathikar, —Soft Computing‖, Narosa Publishing House, New Delhi, 2008

WEB REFERENCES

1. http://mirlab.org/jang/book/

COURSE OUTCOMES

Upon completion of the course, students will be able to

CO1	Understand the implementation of Neural Network algorithms.
CO2	Design solutions for complex problems using Fuzzy set.
CO3	Design and apply Genetic and Classification Algorithms

MAPPING OF COs WITH POs AND PSOs

COs	PROGRAM OUTCOMES (POs)												PROGRAM SPECIFIC OUTCOMES (PSOs)		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	3	3	3	2	-	-	-	-	2	2	2	3	3	3
CO2	3	3	3	3	2	-	-	-	-	2	2	2	3	3	3
CO3	3	3	3	3	2	-	-	-	-	2	2	2	3	3	3

Artificial Intelligence and Data Science

Name of the Lab: DS1707 Neuro-Fuzzy Computing Lab **Staff Name:**

S.No.	Date of Experiment	Name of the Experiment	Date of Submission	Signature
1.		Implementation of Perceptron		
2.		Implementation of Perceptron Rule		
3.		Implementation of Artificial Neural Network		
4.		Implementation of Fuzzy Sets (Membership Functions)		
5.		Implementation of Fuzzy Sets (Standard, Yager, Sugeno Complements, Union & Intersection)		
6.		Data Fitting by Regression		
7.		Implementation of Covariance and Correlation		
8.		Implementation of Crisp Model		
9.		Implementation of Logic Gates		
10.		Implementation of Genetic Algorithms		
11.		Implementation of Classification Algorithm		

Ex.No: 01 IMPLEMENTATION OF PERCEPTRON Date:

Aim:

To implement the Perceptron learning algorithm for an AND gate using MATLAB's built-in perceptron function, and to train, test, and analyze the perceptron's performance.

Algorithm:

Perceptron Learning using MATLAB Built-in Function

1. **Start.**
2. Define input matrix **inputs** and target output vector **targets** for the AND gate.
3. Create a **Perceptron network** using **perceptron**.
4. Configure the network using **configure()** to set input and output sizes.
5. **Manually assign**:
   - Initial weights using **net.IW{1,1}**
   - Bias using **net.b{1}**
   - Learning rate via **net.trainParam.lr**
   - Maximum number of epochs via **net.trainParam.epochs**
6. Train the network using **train(net, inputs, targets).**
7. Test the trained network by computing outputs: **outputs = net(inputs).**
8. Display the final weights, bias, and outputs.
9. **Visualize** the network structure using **view(net).**
10. **Stop.**

Program:

```
clear; clc;
inputs = [0 0 1 1;   % x1
          0 1 0 1];  % x2
targets = [0 0 0 1]; % AND gate target
%% Step 2: Create Perceptron Network
net = perceptron;
% Step 3: Configure the network (this sets input/output sizes)
net = configure(net, inputs, targets);
% Step 4: Manually assign initial weights, bias, learning rate
net.IW{1,1} = [0.3 -0.1];  % 1 neuron, 2 inputs
net.b{1}    = 0.2;       % Bias for single neuron
```

```
net.trainParam.lr = 0.1;  % Learning rate
net.trainParam.epochs = 10;
% Display manually set weights and bias
disp('Initial Weights:');
disp(net.IW{1,1});
disp('Initial Bias:');
disp(net.b{1});
%% Step 5: Train the Network
net = train(net, inputs, targets);
%% Step 6: Test the Network
outputs = net(inputs);
%% Step 7: Display Results
disp('Inputs:');
disp(inputs');
disp('Targets:');
disp(targets');
disp('Outputs:');
disp(outputs');
%% Step 8: View Updated Weights and Bias
disp('Updated Weights after Training:');
disp(net.IW{1,1});
disp('Updated Bias after Training:');
disp(net.b{1});
disp('Updated Learning rate:');
disp(net.trainParam.lr);
% View network diagram
view(net);
```

Outputs:

```
Initial Weights:
   0.3000   -0.1000
Initial Bias:
   0.2000
```

Inputs:

0 0

0 1

1 0

1 1

Targets:

0

0

0

1

Outputs:

0

0

0

1

Updated Weights after Training:

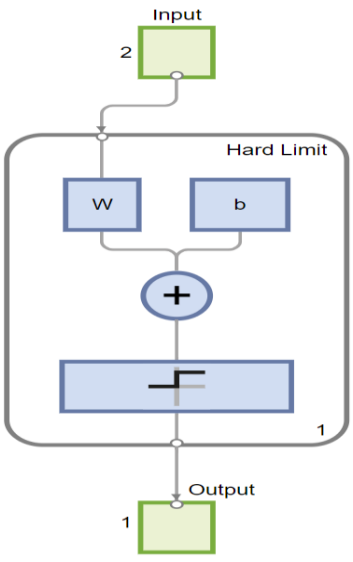
2.3000 0.9000

Updated Bias after Training:

-2.8000

Updated Learning rate:

0.1000



Network Diagram

Network Diagram

Training Results

Training finished: Met performance criterion

Training Progress

Unit	Initial Value	Stopped Value	Target Value
Epoch	0	5	10
Elapsed Time	-	00:00:00	-
Performance	0.75	0	0

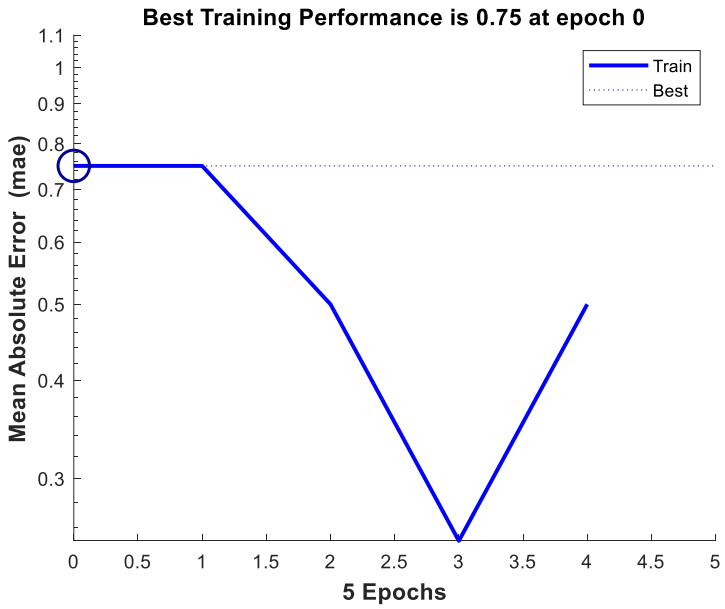
Training Algorithms

Data Division: Training Only dividetrain
Training: Cyclical Weight/Bias Rule trainc
Performance: Mean Absolute Error mae
Calculations: MATLAB

Training Plots

Performance | Training State
Confusion

Training Results



Performance Results

Results:

The perceptron model was successfully implemented using MATLAB's built-in perceptron function. After training, the model correctly learned the AND logic gate behavior and classified all input patterns accurately.

Ex.No: 02 IMPLEMENTATION OF PERCEPTRON RULE Date:

Aim:

To implement a perceptron model for simulating the behavior of the AND gate, train it using the perceptron learning rule, and plot the decision boundary in MATLAB.

Algorithm

Perceptron Learning Algorithm for AND Gate

1. **Start**
2. Define input patterns X and target outputs T for the AND gate.
3. Initialize:
   - Weights W
   - Bias b
   - Learning rate α
   - Maximum number of epochs
4. **Repeat until convergence or max epochs:**
   - For each input pattern:
     - Compute weighted sum $z = W \cdot X + b$
     - Apply activation function (step function): $y = 1$ if $z \geq 0$ else 0
     - Compute error e = target - y
     - **If error ≠ 0**, update:
       - ✓ $W = W + \alpha \times e \times X$
       - ✓ $b = b + \alpha \times e$
   - If no errors in the epoch → training is complete
5. Plot the decision boundary using the final weights and bias.
6. Test the trained model on all inputs.
7. **Stop**

Program:

```
Clear; clc;
%% AND Gate Perceptron Implementation
% Step 1: Input and Output (Target) for AND Gate
X = [0 0; 0 1; 1 0; 1 1];      % Inputs (4 samples, 2 inputs)
T = [0; 0; 0; 1];              % AND gate targets
% Step 2: Initialize weights, bias, and learning rate
```

```
W = [0.3, -0.1];            % Initial weights (2 inputs)
b = 0.2;                    % Initial bias
alpha = 0.1;                % Learning rate
% Step 3: Training loop
max_epochs = 20;
fprintf ("Initial Weights: %.2f %.2f | Bias: %.2f\n", W(1), W(2), b);
for epoch = 1:max_epochs
   error_count = 0;
   fprintf('\nEpoch %d\n', epoch);
   for i = 1:4
      x = X(i, :);
      target = T(i);
      % Perceptron output (step activation)
      z = dot(W, x) + b;
      y = z >= 0;   % Step function
      % Compute error
      e = target - y;
      % Update weights and bias if error occurs
      if e ~= 0
         W = W + alpha * e * x;
         b = b + alpha * e;
         error_count = error_count + 1;
      end
      fprintf('Input: [%d %d] | Target: %d | Output: %d | Updated W: [%.2f %.2f] | b: %.2f\n',
x(1), x(2), target, y, W(1), W(2), b);
   end
   % Stop training if no error
   if error_count == 0
      disp('Training completed successfully.');
      break;
   end
end
% Final result
```

```
fprintf('\nFinal Weights: %.2f %.2f | Final Bias: %.2f\n', W(1), W(2), b);
% Test the trained perceptron
fprintf('\nTesting Trained Perceptron:\n');
for i = 1:4
   x = X(i, :);
   y = (dot(W, x) + b) >= 0;
   fprintf('Input: [%d %d] | Output: %d\n', x(1), x(2), y);
end
% Plotting the decision boundary
figure;
hold on;
grid on;
title('Perceptron Decision Boundary for AND Gate');
xlabel('x1');
ylabel('x2');
% Plot original data points
for i = 1:4
   if T(i) == 1
      h1=plot(X(i,1), X(i,2), 'go', 'MarkerSize', 10, 'LineWidth', 2); % Green for class 1
   else
      h2=plot(X(i,1), X(i,2), 'rx', 'MarkerSize', 10, 'LineWidth', 2); % Red for class 0
   end
end
% Create a line for the decision boundary: w1*x1 + w2*x2 + b = 0
x1_vals = linspace(-0.5, 1.5, 100);
x2_vals = -(W(1) * x1_vals + b) / W(2);  % Solve for x2
% Plot decision boundary
h3=plot(x1_vals, x2_vals, 'b-', 'LineWidth', 2);
legend([h1 h2 h3], {'Class 1 (Output = 1)', 'Class 0 (Output = 0)', 'Decision Boundary'},
'Location', 'northeast');
axis([-0.5 1.5 -0.5 1.5]);
hold off;
```

Outputs:

Initial Weights: 0.30 -0.10 | Bias: 0.20

Epoch 1

Input: [0 0] | Target: 0 | Output: 1 | Updated W: [0.30 -0.10] | b: 0.10

Input: [0 1] | Target: 0 | Output: 1 | Updated W: [0.30 -0.20] | b: 0.00

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 -0.20] | b: -0.10

Input: [1 1] | Target: 1 | Output: 0 | Updated W: [0.30 -0.10] | b: 0.00

Epoch 2

Input: [0 0] | Target: 0 | Output: 1 | Updated W: [0.30 -0.10] | b: -0.10

Input: [0 1] | Target: 0 | Output: 0 | Updated W: [0.30 -0.10] | b: -0.10

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 -0.10] | b: -0.20

Input: [1 1] | Target: 1 | Output: 0 | Updated W: [0.30 0.00] | b: -0.10

Epoch 3

Input: [0 0] | Target: 0 | Output: 0 | Updated W: [0.30 0.00] | b: -0.10

Input: [0 1] | Target: 0 | Output: 0 | Updated W: [0.30 0.00] | b: -0.10

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 0.00] | b: -0.20

Input: [1 1] | Target: 1 | Output: 0 | Updated W: [0.30 0.10] | b: -0.10

Epoch 4

Input: [0 0] | Target: 0 | Output: 0 | Updated W: [0.30 0.10] | b: -0.10

Input: [0 1] | Target: 0 | Output: 1 | Updated W: [0.30 0.00] | b: -0.20

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 0.00] | b: -0.30

Input: [1 1] | Target: 1 | Output: 0 | Updated W: [0.30 0.10] | b: -0.20

Epoch 5

Input: [0 0] | Target: 0 | Output: 0 | Updated W: [0.30 0.10] | b: -0.20

Input: [0 1] | Target: 0 | Output: 0 | Updated W: [0.30 0.10] | b: -0.20

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 0.10] | b: -0.30

Input: [1 1] | Target: 1 | Output: 0 | Updated W: [0.30 0.20] | b: -0.20

Epoch 6

Input: [0 0] | Target: 0 | Output: 0 | Updated W: [0.30 0.20] | b: -0.20

Input: [0 1] | Target: 0 | Output: 0 | Updated W: [0.30 0.20] | b: -0.20

Input: [1 0] | Target: 0 | Output: 1 | Updated W: [0.20 0.20] | b: -0.30

Input: [1 1] | Target: 1 | Output: 1 | Updated W: [0.20 0.20] | b: -0.30

Epoch 7

Input: [0 0] | Target: 0 | Output: 0 | Updated W: [0.20 0.20] | b: -0.30

Input: [0 1] | Target: 0 | Output: 0 | Updated W: [0.20 0.20] | b: -0.30

Input: [1 0] | Target: 0 | Output: 0 | Updated W: [0.20 0.20] | b: -0.30

Input: [1 1] | Target: 1 | Output: 1 | Updated W: [0.20 0.20] | b: -0.30

Training completed successfully.

Final Weights: 0.20 0.20 | Final Bias: -0.30

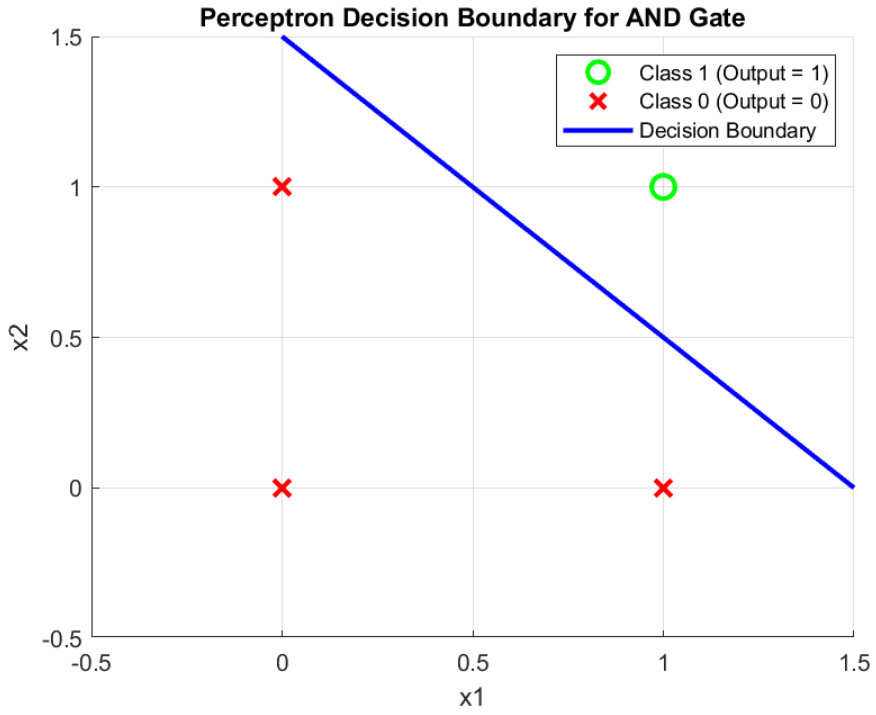
Testing Trained Perceptron:

Input: [0 0] | Output: 0

Input: [0 1] | Output: 0

Input: [1 0] | Output: 0

Input: [1 1] | Output: 1



Results:

The perceptron was successfully implemented and trained to model the behavior of the AND logic gate.

- The initial weights and bias were updated using the **Perceptron Learning Rule**.
- The training converged within a finite number of epochs.
- The final learned weights and bias accurately classified all input patterns of the AND gate.

Ex.No: 03 IMPLEMENTATION OF ARTIFICIAL NEURAL NETWORK Date:

Aim:

To design, train, and evaluate a Feedforward Neural Network using the **bodyfat_dataset** to predict body fat percentage from physiological measurements.

MATLAB Inbuilt Datasets:

Dataset Name	Function to Load	Description
bodyfat_dataset	**[X,T] = bodyfat_dataset;**	**Body fat prediction from body measurements**
cancer_dataset	[X,T] = cancer_dataset;	Breast cancer diagnosis data (benign/malignant)
carbig	load carbig	Car performance data (MPG, horsepower, etc.)
fisheriris	load fisheriris	Iris flower classification (classic dataset)
abalone_dataset	[X,T] = abalone_dataset;	Predict age of abalone from physical measurements
ionosphere	load ionosphere	Radar data for binary classification
simplefit_dataset	[X,T] = simplefit_dataset;	Simple function fitting (X→T)
house_dataset	[X,T] = house_dataset;	Predict house prices from features
maglev_dataset	[X,T] = maglev_dataset;	Magnetic levitation control data
chemical_dataset	[X,T] = chemical_dataset;	Chemical process data for regression
engine_dataset	[X,T] = engine_dataset;	Engine condition monitoring data
ph_dataset	[X,T] = ph_dataset;	pH neutralization process modeling
sine_dataset	[X,T] = sine_dataset;	Sine wave data for time series prediction
mushroom_dataset	[X,T] = mushroom_dataset;	Classification of mushrooms (edible or poisonous)
digittrain_dataset	[X,T] = digittrain_dataset;	Handwritten digit images for classification

Change Training Algorithm

MATLAB uses **trainFcn** to specify the training algorithm.

Example: net.trainFcn = 'traingd'; **% Gradient Descent**

TrainFcn	Description
'trainc'	Perceptron learning rule (default)
'traingd'	Gradient Descent
'traingdm'	Gradient Descent with Momentum
'traingdx'	GD with Momentum and Adaptive LR
'trainlm'	Levenberg-Marquardt (default)
'trainbr'	Bayesian Regularization
'trainrp'	Resilient Backpropagation

Change Performance Function

Set **net.performFcn** to define how performance is calculated during training.

Example: net.performFcn = 'mse'; % **Mean Squared Error**

PerformFcn	Description
'mse'	Mean Squared Error (default)
'mae'	Mean Absolute Error
'sse'	Sum Squared Error
'crossentropy'	For classification tasks

Set Training Parameters

net.trainParam.lr = 0.05; **% Learning Rate**

net.trainParam.epochs = 200; **% Max number of training iterations**

net.trainParam.goal = 0.01**; % Performance goal**

Customize Data Division

By default, data is split into training/validation/testing sets.

Example: net.divideFcn = 'dividerand'; **% Random division (default)**

divideFcn	Description
'dividerand'	Random (default)
'divideblock'	Blocks of sequential data
'divideind'	Manual index-based division
'divideint'	Interleaved blocks
'dividetrain'	Use all data for training only

To manually control the percentages

net.divideParam.trainRatio = 0.7;

net.divideParam.valRatio = 0.15;

net.divideParam.testRatio = 0.15;

MATLAB Transfer Functions (Activation)

Example: net.layers{i}.transferFcn = 'function_name';

Function Name	Full Name	Output Range	Use Case
'logsig'	Logistic Sigmoid	(0, 1)	Binary classification, hidden/output
'tansig'	Hyperbolic Tangent Sigmoid	(−1, 1)	Hidden layers, faster convergence
'purelin'	Linear	(−∞, ∞)	Regression, output layer
'hardlim'	Hard Limit (binary step)	0 or 1	Classic perceptron
'hardlims'	Symmetric Hard Limit	−1 or 1	Simple symmetric binary networks
'satlin'	Saturating Linear (0 to 1)	[0, 1]	Clipping activations
'satlins'	Symmetric Saturating Linear	[−1, 1]	Symmetric clipped outputs
'poslin'	ReLU (Rectified Linear Unit)	[0, ∞)	Deep learning (hidden layers)
'radbas'	Radial Basis	(0, 1)	RBF networks (hidden units)
'softmax'	Softmax	(0, 1), sums to 1	Classification (output layer)
'tribas'	Triangular Basis	(0, 1)	Specialized pattern recognition
'compet'	Competitive	binary vector	Winner-takes-all networks

The Problem: Estimate Body Fat Percentage:

To build a neural network that can estimate the body fat percentage of a person described by thirteen physical attributes:

- Age (years)
- Weight (lbs)
- Height (inches)
- Neck circumference (cm)

- Chest circumference (cm)
- Abdomen circumference (cm)
- Hip circumference (cm)
- Thigh circumference (cm)
- Knee circumference (cm)
- Ankle circumference (cm)
- Biceps (extended) circumference (cm)
- Forearm circumference (cm)
- Wrist circumference (cm)

Algorithm:

1. **Load Dataset**
   - Use the built-in bodyfat_dataset which provides input features (X) and target values (T).
2. **Divide Dataset**
   - Split the dataset into training (70%), validation (15%), and testing (15%) using dividerand.
3. **Create Feedforward Network**
   - Initialize a feedforward neural network using feedforwardnet with one hidden layer of size 10.
4. **Set Data Division**
   - Assign training, validation, and test indices using divideind.
5. **Train the Network**
   - Train the model using train() function.
   - Internally uses backpropagation and gradient descent.
6. **Evaluate Performance**
   - Compute Mean Squared Error (MSE) on entire dataset and specifically on test data.
7. **Visualize Results**
   - Plot:
     - Performance curve
     - Regression plot
     - Training state

- Error histogram

8. Compare Outputs

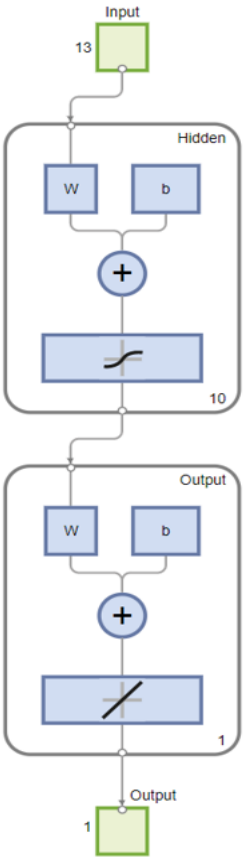
- Display actual vs predicted output values for test data samples.

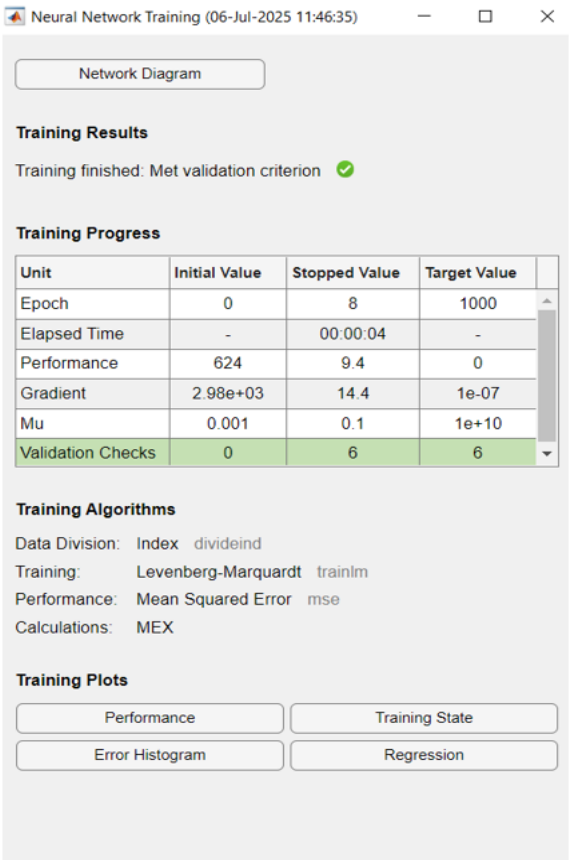
Program:

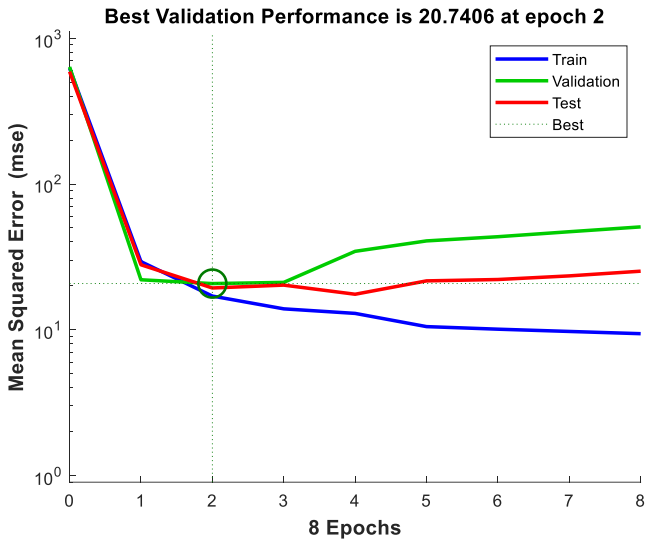
```
clc;
clear;
% Load the body fat dataset
[X, T] = bodyfat_dataset;
% Split into training, validation, and test sets (default is 70/15/15)
[trainInd, valInd, testInd] = dividerand(size(X,2), 0.7, 0.15, 0.15);
% Create a feedforward neural network
hiddenLayerSize = 10;
net = feedforwardnet(hiddenLayerSize);
% Set training, validation, test division manually
net.divideFcn = 'divideind';
net.divideParam.trainInd = trainInd;
net.divideParam.valInd = valInd;
net.divideParam.testInd = testInd;
% Train the network
[net, tr] = train(net, X, T);
% View the network
view(net);
% Test the network
Y = net(X);
perf = perform(net, T, Y);
% Display performance
fprintf('Overall MSE performance: %.4f\n', perf);
% Plot performance
figure, plotperform(tr);
% Plot regression
figure, plotregression(T, Y);
% Plot training state
```

```
figure, plottrainstate(tr);
% Plot error histogram
figure, ploterrhist(T - Y);
% Predict on test samples only
X_test = X(:, tr.testInd);
T_test = T(:, tr.testInd);
Y_test = net(X_test);
test_perf = perform(net, T_test, Y_test);
fprintf('Test MSE: %.4f\n', test_perf);
% Optional: display actual vs predicted
disp('Actual vs Predicted for Test Samples:');
disp(table(T_test', Y_test', 'VariableNames', {'Actual', 'Predicted'}));
```

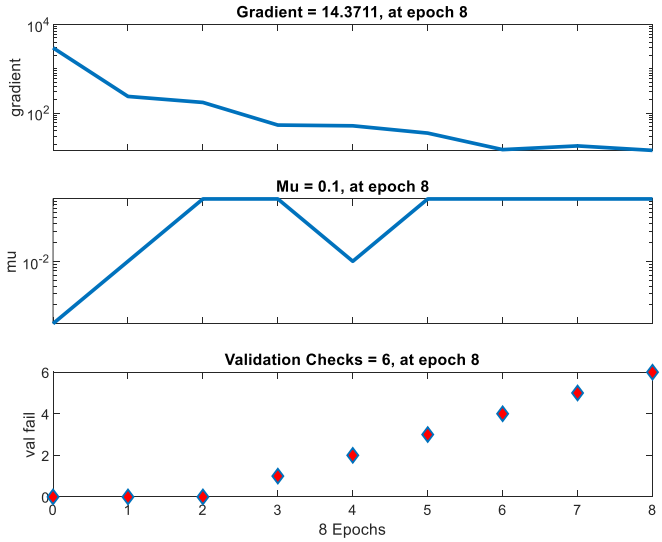
Output:

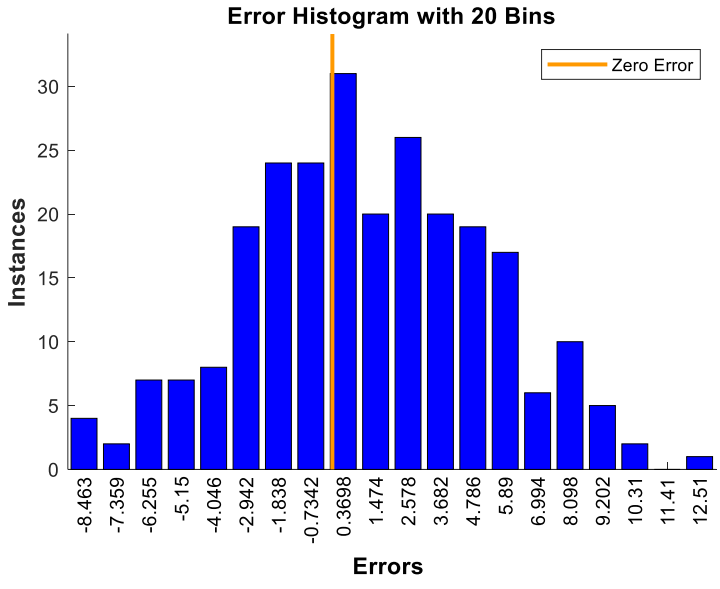


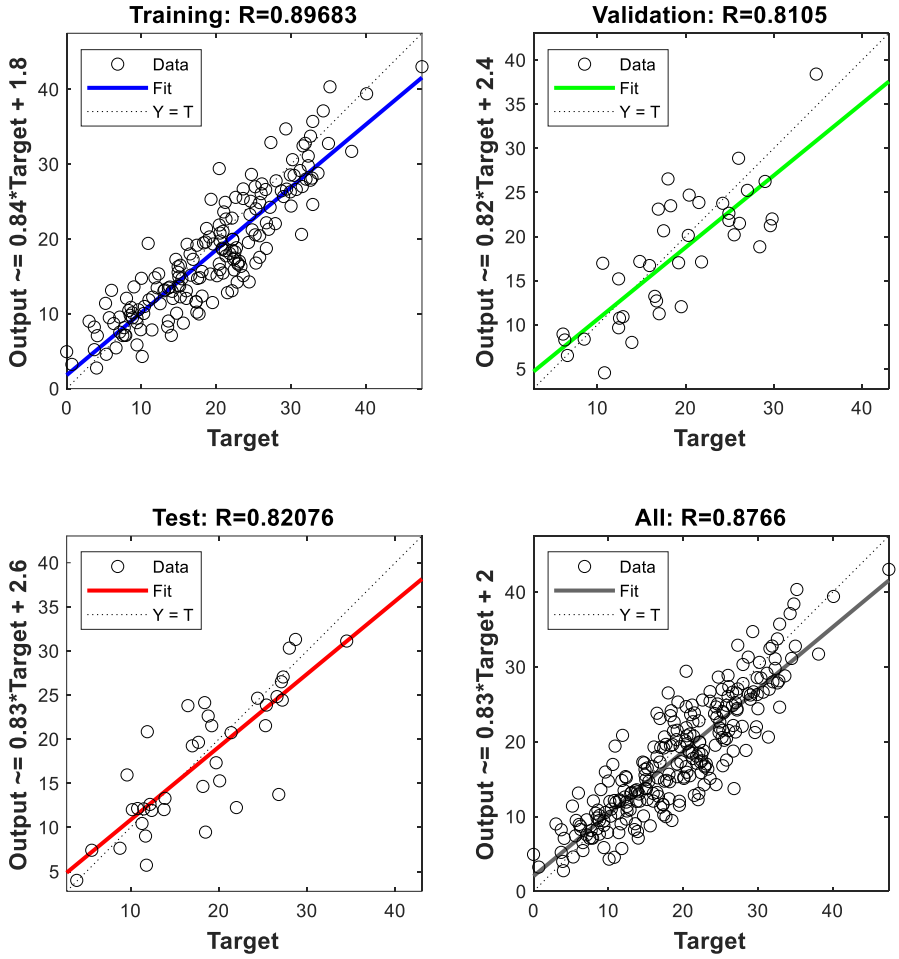




Training State







Overall MSE performance: 17.9338

Test MSE: 19.3455

Result:

The feedforward neural network was successfully trained using the bodyfat_dataset. The architecture consisted of a single hidden layer with 10 neurons. During evaluation, the network achieved an overall Mean Squared Error (MSE) of approximately **17.9338**, and the test set MSE was **19.3455**, indicating a good fit to the data.

Ex.No: 04 **IMPLEMENTATION OF FUZZY SETS** **Date:**

(MEMBERSHIP FUNCTIONS)

Aim:

To implement different fuzzy membership functions (Triangular, Trapezoidal, Gaussian, and Bell-shaped) using MATLAB and to visualize their behavior.

Theory:

Fuzzy Membership Function

A membership function (MF) is a curve that defines how each element in the universe of discourse is mapped to a membership value between 0 and 1.

It represents the degree of truth or degree of belonging of an element to a fuzzy set.

Formally, if X is the universe of discourse, a fuzzy set A is characterized by a membership function:

$$\mu_A : X \to [0,1]$$

Where for each $x \in X$, the function $\mu_A(x)$ gives the membership grade of x in fuzzy set A.

Common Types of Membership Functions:

Triangular Membership Function

$$\mu_A(x) = \begin{cases} 0 & x \le a \text{ or } x \ge c \\ \frac{x-a}{b-a} & a \le x \le b \\ \frac{c-x}{c-b} & b \le x \le c \end{cases}$$

MATLAB: mu = trimf(x,[a b c]);

Trapezoidal Membership Function

$$\mu_A(x) = \begin{cases} 0 & x \le a \text{ or } x \ge d \\ \frac{x-a}{b-a} & a \le x \le b \\ 1 & b \le x \le c \\ \frac{d-x}{d-c} & c \le x \le d \end{cases}$$

MATLAB: mu = trapmf(x,[a b c d]);

Gaussian Membership Function

$$\mu_A(x) = e^{-\frac{(x-c)^2}{2\sigma^2}}$$

Where c is the center and σ is the width.

MATLAB: mu = gaussmf(x,[sigma c]);

Bell-Shaped Membership Function

$$\mu_A(x) = \frac{1}{1 + \left|\frac{x-c}{a}\right|^{2b}}$$

MATLAB: mu = gbellmf(x,[a b c]);

Algorithm:

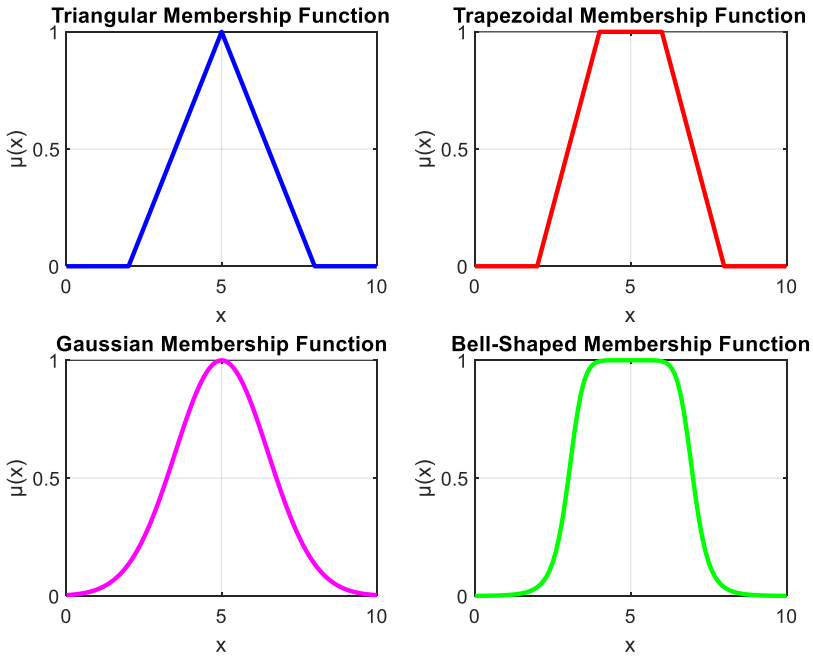
1. Start.
2. Define the universe of discourse x $(e.g., x = 0:0.1:10)$.
3. Select the type of membership function (**Triangular, Trapezoidal, Gaussian, or Bell**).
4. Define parameters of the membership function.
5. Use MATLAB functions (**trimf, trapmf, gaussmf, gbellmf**) to generate the MFs.
6. Plot each membership function with proper labels.
7. Stop.

Program:

```
clc;
clear;
x = 0:0.1:10;
% Triangular MF
mu_tri = trimf(x, [2 5 8]);
% Trapezoidal MF
mu_trap = trapmf(x, [2 4 6 8]);
% Gaussian MF
mu_gauss = gaussmf(x, [1.5 5]);
% Bell-Shape MF
mu_bell = gbellmf(x, [2 4 5]);
figure (1);
subplot(2,2,1);
plot(x, mu_tri, 'b', 'LineWidth', 2);
title('Triangular Membership Function');
xlabel('x'); ylabel('μ(x)');
grid on;
subplot(2,2,2);
plot(x, mu_trap, 'r', 'LineWidth', 2);
```

```
title('Trapezoidal Membership Function');
xlabel('x'); ylabel('μ(x)');
grid on;
subplot(2,2,3);
plot(x, mu_gauss, 'm', 'LineWidth', 2);
title('Gaussian Membership Function');
xlabel('x'); ylabel('μ(x)');
grid on;
subplot(2,2,4);
plot(x, mu_bell, 'g', 'LineWidth', 2);
title('Bell-Shaped Membership Function');
xlabel('x'); ylabel('μ(x)');
grid on;
```

Output:



Results:

Different membership functions were successfully implemented in MATLAB. The triangular and trapezoidal membership functions exhibited piecewise linear behavior. The Gaussian membership function produced a smooth bell-shaped curve controlled by its mean and standard deviation. The generalized bell membership function provided a flexible curve with adjustable slope and width. Hence, the behavior of fuzzy sets with different membership functions was visualized successfully.

Ex.No: 05 **IMPLEMENTATION OF FUZZY SETS** **Date:**

(STANDARD, YAGER, SUGENO COMPLEMENTS, UNION & INTERSECTION)

Aim:

To implement fuzzy sets and perform fuzzy set operations such as union, intersection, standard complement, Yager complement, and Sugeno complement using MATLAB.

Theory:

Let A and B be fuzzy sets in universe of discourse $\boldsymbol{X}$, with membership functions $\mu_A(x)$ and $\mu_B(x)$.

Union (OR):

$$\mu_{A\cup B}(x) = \max(\mu_A(x), \mu_B(x))$$

Intersection (AND):

$$\mu_{A\cap B}(x) = \min(\mu_A(x), \mu_B(x))$$

Standard Complement:

$$\mu_{\neg A}(x) = 1 - \mu_A(x)$$

Yager Complement ($\boldsymbol{parameter\ p > 0}$):

$$\mu^{Y}_{\neg A}(x) = \left(1 - \mu_A(x)^p\right)^{\frac{1}{p}}$$

Sugeno Complement ($\boldsymbol{parameter\ a > -1}$):

$$\mu^{S}_{\neg A}(x) = \frac{1 - \mu_A(x)}{1 + a \cdot \mu_A(x)}$$

Algorithm:

1. Start.
2. Define the universe of discourse $(e.g., x = 0{:}\,0.1{:}\,10)$.
3. Define fuzzy sets A and B using Gaussian membership functions.
4. Compute:
   - Standard complement: $1 - \mu(x)$
   - Yager complement with chosen p
   - Sugeno complement with chosen a

- Union using max ()
- Intersection using min ()

5. Plot fuzzy sets and their operations using subplot for clarity.
6. Stop.

Program:

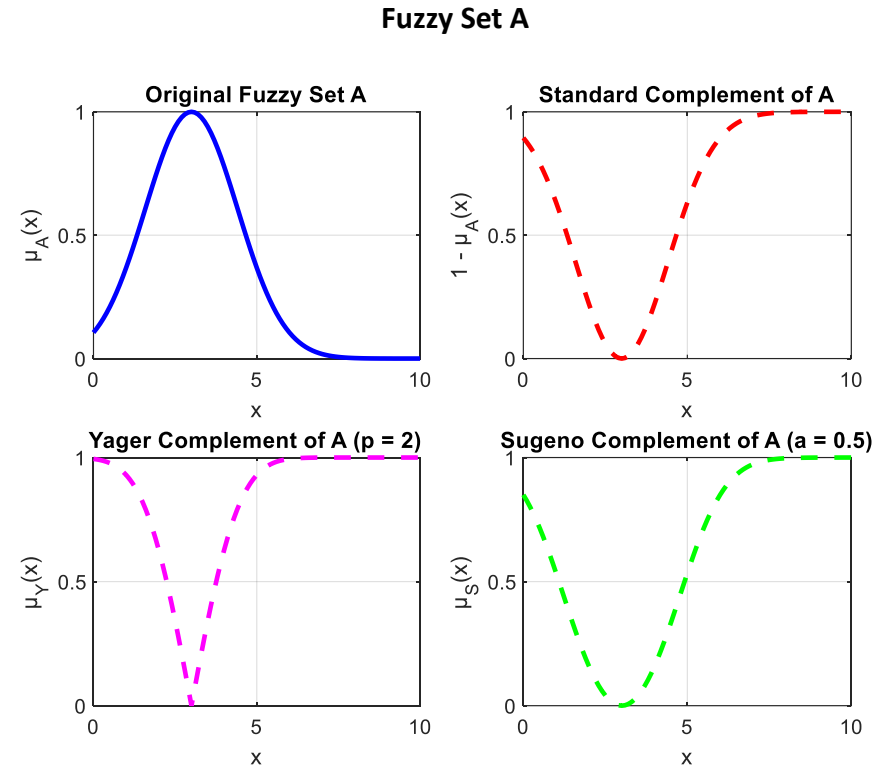
```
clc;
clear;
% Step 1: Define the universe of discourse
x = 0:0.1:10;
% Step 2: Define fuzzy membership functions
% Fuzzy Set A: Gaussian around 3
%mu_A = exp(-((x - 3).^2) / 4);
mu_A = gaussmf(x, [sqrt(2) 3]);
% Fuzzy Set B: Gaussian around 7
%mu_B = exp(-((x - 7).^2) / 4);
mu_B = gaussmf(x, [sqrt(2) 7]);
% Step 3: Fuzzy operations
% Complement of A
mu_complement_A = 1 - mu_A;
% Complement of B
mu_complement_B = 1 - mu_B;
% Yager Complement of A
p = 2;
mu_complement_yager_A = (1 - mu_A.^p).^(1/p);
% Yager Complement of B
p = 2;
mu_complement_yager_B = (1 - mu_B.^p).^(1/p);
% Sugeno Complement of A
a = 0.5;
mu_complement_sugeno_A = (1 - mu_A) ./ (1 + a * mu_A);
% Sugeno Complement of B
a = 0.5
```

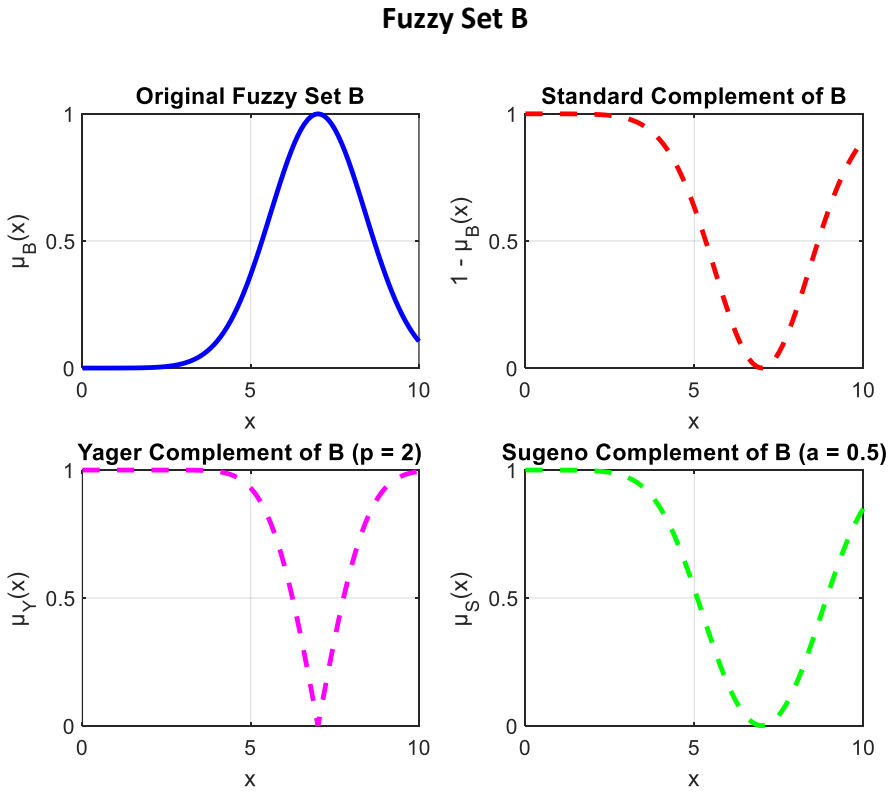
```
mu_complement_sugeno_B = (1 - mu_B) ./ (1 + a * mu_B);
% Union (max)
mu_union = max(mu_A, mu_B);
% Intersection (min)
mu_intersection = min(mu_A, mu_B);
figure (1);
subplot(2,2,1);
plot(x, mu_A, 'b', 'LineWidth', 2);
title('Original Fuzzy Set A');
xlabel('x'); ylabel('μ_A(x)');
grid on;
subplot(2,2,2);
plot(x, mu_complement_A, 'r--', 'LineWidth', 2);
title('Standard Complement of A');
xlabel('x'); ylabel('1 - μ_A(x)');
grid on;
subplot(2,2,3);
plot(x, mu_complement_yager_A, 'm--', 'LineWidth', 2);
title(['Yager Complement of A (p = ' num2str(p) ')']);
xlabel('x'); ylabel('μ_Y(x)');
grid on;
subplot(2,2,4);
plot(x, mu_complement_sugeno_A, 'g--', 'LineWidth', 2);
title(['Sugeno Complement of A (a = ' num2str(a) ')']);
xlabel('x'); ylabel('μ_S(x)');
grid on;
figure (2);
subplot(2,2,1);
plot(x, mu_B, 'b', 'LineWidth', 2);
title('Original Fuzzy Set B');
xlabel('x'); ylabel('μ_B(x)');
grid on;
```

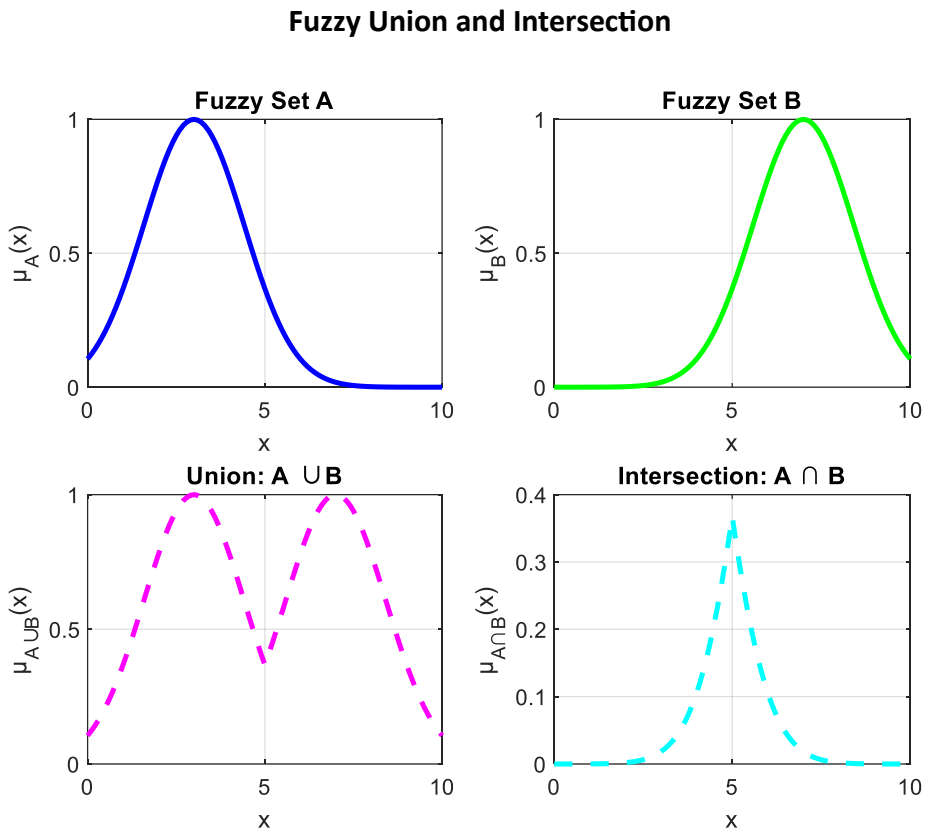
```
subplot(2,2,2);
plot(x, mu_complement_B, 'r--', 'LineWidth', 2);
title('Standard Complement of B');
xlabel('x'); ylabel('1 - μ_B(x)');
grid on;
subplot(2,2,3);
plot(x, mu_complement_yager_B, 'm--', 'LineWidth', 2);
title(['Yager Complement of B (p = ' num2str(p) ')']);
xlabel('x'); ylabel('μ_Y(x)');
grid on;
subplot(2,2,4);
plot(x, mu_complement_sugeno_B, 'g--', 'LineWidth', 2);
title(['Sugeno Complement of B (a = ' num2str(a) ')']);
xlabel('x'); ylabel('μ_S(x)');
grid on;
figure (3);
subplot(2,2,1);
plot(x, mu_A, 'b', 'LineWidth', 2);
title('Fuzzy Set A');
xlabel('x'); ylabel('μ_A(x)');
grid on;
subplot(2,2,2);
plot(x, mu_B, 'g', 'LineWidth', 2);
title('Fuzzy Set B');
xlabel('x'); ylabel('μ_B(x)');
grid on;
subplot(2,2,3);
plot(x, mu_union, '--m', 'LineWidth', 2);
title('Union: A ∪ B');
xlabel('x'); ylabel('μ_{A∪B}(x)');
grid on;
subplot(2,2,4);
plot(x, mu_intersection, '--c', 'LineWidth', 2);
```

```
title('Intersection: A ∩ B');
xlabel('x'); ylabel('μ_{A∩B}(x)');
grid on;
```

Output:







Result:

Fuzzy sets and their operations (union, intersection, standard complement, Yager complement, and Sugeno complement) were successfully implemented and visualized using MATLAB.

Ex.No: 06 **DATA FITTING BY REGRESSION** **Date:**

Aim:

To implement data fitting using regression analysis in MATLAB and to visualize the best-fit line for a given dataset.

Theory:

Regression analysis is a statistical technique used to model and analyze the relationship between dependent (Y) and independent (X) variables.

Linear Regression (First-Order Polynomial):

$$Y = mX + c$$

$m = Slope$

$c = Intercept$

The coefficients m, c are obtained by minimizing the **sum of squared errors** between actual and predicted values.

Polynomial Regression (Higher-Order):

$$Y = a_0 + a_1X + a_2X^2 + \cdots + a_nX^n$$

In MATLAB, regression fitting can be done using:

$polyfit(X, Y, n) \rightarrow fits\ a\ polynomial\ of\ degree\ n.$

$polyval(p, X) \rightarrow evaluates\ fitted\ values\ at\ points\ X.$

Algorithm:

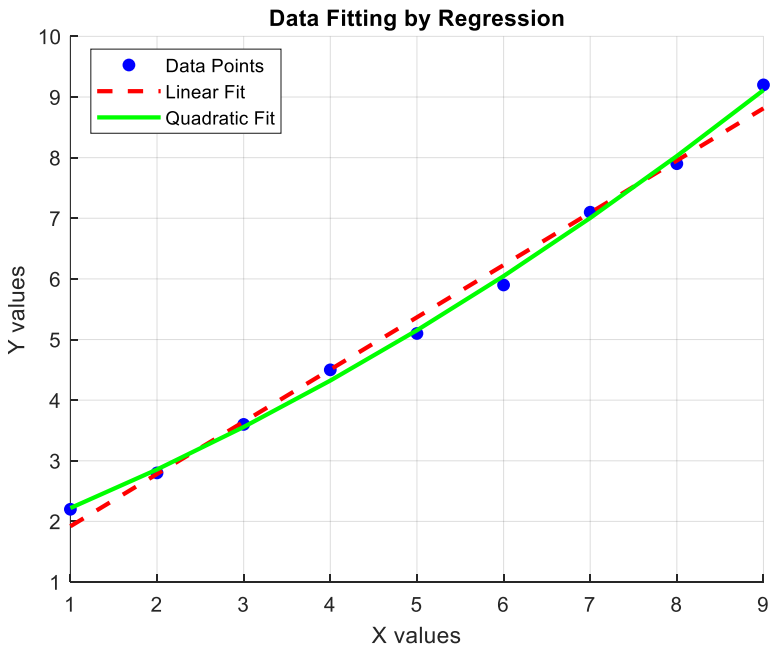
1. Start.
2. Input the dataset X, Y.
3. Choose the degree of regression (linear, quadratic, etc.).
4. Use MATLAB polyfit to compute regression coefficients.
5. Use polyval to estimate fitted Y values.
6. Plot the original data points and the fitted regression curve.
7. Stop.

Program:

```
clc;
clear;
% Step 1: Define dataset
X = [1 2 3 4 5 6 7 8 9];
```

```
Y = [2.2 2.8 3.6 4.5 5.1 5.9 7.1 7.9 9.2];
% Step 2: Linear regression (1st order polynomial)
p1 = polyfit(X, Y, 1);      % slope & intercept
Y_fit1 = polyval(p1, X);
% Step 3: Quadratic regression (2nd order polynomial)
p2 = polyfit(X, Y, 2);
Y_fit2 = polyval(p2, X);
% Step 4: Plot data and regression lines
figure;
scatter(X, Y, 'b', 'filled'); hold on;
plot(X, Y_fit1, 'r--', 'LineWidth', 2);
plot(X, Y_fit2, 'g-', 'LineWidth', 2);
title('Data Fitting by Regression');
xlabel('X values'); ylabel('Y values');
legend('Data Points','Linear Fit','Quadratic Fit','Location','best');
grid on;
```

Output:



Result:

Data fitting using regression was successfully implemented in MATLAB. Both linear and quadratic regression models were fitted to the dataset. The regression plots demonstrated that the quadratic model provided a closer fit to the data compared to the linear model.

Ex.No: 07 **IMPLEMENTATION OF COVARIANCE AND CORRELATION** **Date:**

Aim:

To implement covariance and correlation between two datasets using MATLAB and to analyze their linear relationship.

Theory:

Covariance

- **Covariance** is a statistical measure of the degree to which two random variables change together.
- If $X = \{x_1, x_2, \ldots, x_n\}$ and $Y = \{y_1, y_2, \ldots, y_n\}$, the covariance is defined as:

$$\mathrm{Cov}(X,Y) = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{X})(y_i - \bar{Y})$$

Where,

- $\bar{X} = mean\ of\ X$
- $\bar{Y} = mean\ of\ Y$
- $n = Number\ of\ Observations$

Interpretation:

$\boldsymbol{Cov(X,Y) > 0}$ → Positive relationship (both variables increase together).

$\boldsymbol{Cov(X,Y) < 0}$ → Negative relationship (one increases, the other decreases).

$\boldsymbol{Cov(X,Y) = 0}$ → No linear relationship.

Correlation

- The Pearson correlation coefficient is the normalized form of covariance:

$$r_{XY} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

Where,

- $\sigma_X, \sigma_Y\ are\ the\ standard\ deviations\ of\ X\ and\ Y$
- $-1 \leq r_{XY} \geq 1$

Interpretation:

$\boldsymbol{r = +1}$ → Perfect positive correlation

$\boldsymbol{r = -1}$ → Perfect negative correlation

$\boldsymbol{r = 0}$ → No linear correlation

Algorithm:

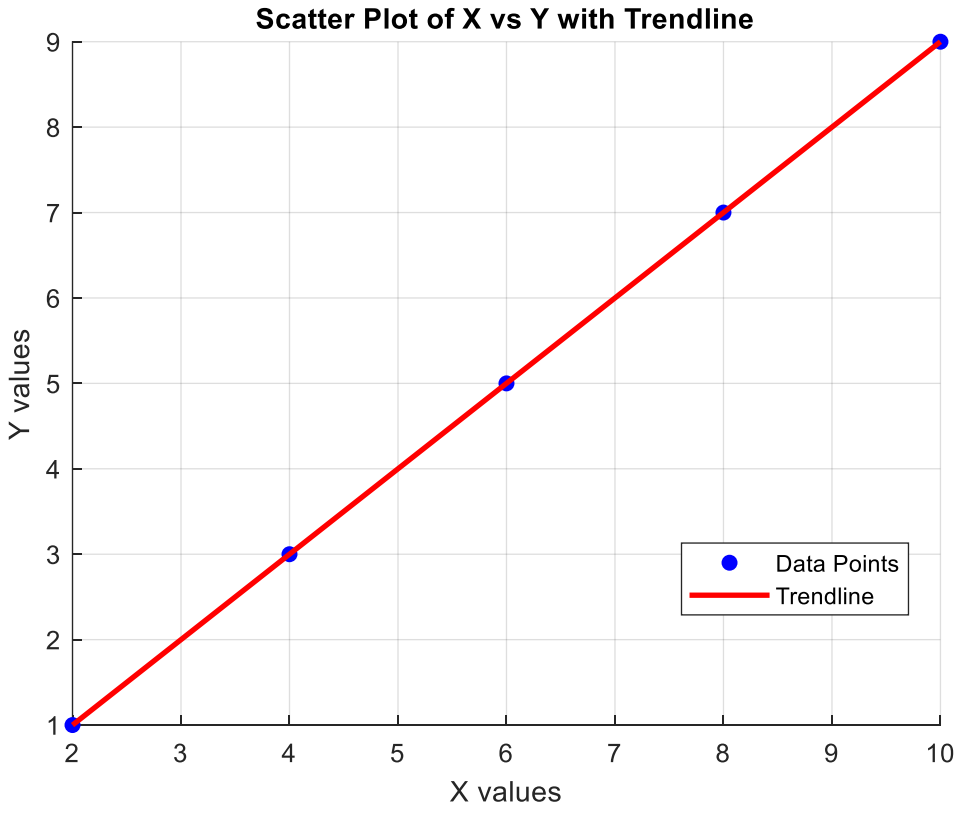
1. Start.
2. Input two datasets X and Y.
3. Compute the means of X and Y
4. Compute covariance using the formula.
5. Compute correlation using covariance and standard deviations.
6. Display results and plot scatter with trendline.
7. Stop.

Program:

```
clc;
clear;
% Step 1: Define datasets
X = [2 4 6 8 10];
Y = [1 3 5 7 9];
% Step 2: Covariance (manual)
n = length(X);
mean_X = mean(X);
mean_Y = mean(Y);
cov_XY = sum((X - mean_X) .* (Y - mean_Y)) / (n - 1);
% Step 3: Standard deviations
std_X = std(X);
std_Y = std(Y);
% Step 4: Correlation (manual)
corr_XY = cov_XY / (std_X * std_Y);
% Step 5: Display results
disp(['Covariance = ', num2str(cov_XY)]);
disp(['Correlation = ', num2str(corr_XY)]);
% Step 6: Cross-check using MATLAB functions
cov_matrix = cov(X, Y);
corr_matrix = corrcoef(X, Y);
disp('Covariance matrix:');
disp(cov_matrix);
```

```
disp('Correlation matrix:');
disp(corr_matrix);
% Step 7: Scatter plot with trendline
figure;
scatter(X, Y, 'filled','b');
hold on;
p = polyfit(X, Y, 1);    % linear fit
y_fit = polyval(p, X);
plot(X, y_fit, 'r', 'LineWidth', 2);
title('Scatter Plot of X vs Y with Trendline');
xlabel('X values');
ylabel('Y values');
legend('Data Points','Trendline','Location','best');
grid on;
```

Output:



Covariance = 10

Correlation = 1

Covariance matrix:

10 10

10 10

Correlation matrix:

1.0000 1.0000

1.0000 1.0000

Results:

The covariance and correlation between the given datasets were successfully computed in MATLAB. The covariance value of **10** indicates a positive relationship. The correlation coefficient of **1** shows a perfect positive linear relationship between X and Y. The scatter plot with a trendline confirmed this linear dependency.

Ex.No: 08 **IMPLEMENTATION OF CRISP MODEL** **Date:**

Aim:

To implement and study the operations of **crisp sets** (union, intersection, difference, and complement) using MATLAB.

Theory:

A **crisp set** is a collection of well-defined distinct objects, where an element either **belongs** to the set or **does not belong**.

The membership function of a crisp set is binary:

$$\mu_A(x) = \begin{cases} 1, & x \in A \\ 0, & x \notin A \end{cases}$$

Crisp Set Operations:

Union:

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

Intersection:

$$A \cap B = \{x \mid x \in A \text{ and } x \in B\}$$

Difference:

$$A - B = \{x \mid x \in A \text{ and } x \notin B\}$$

Complement:

$$\bar{A} = \{x \mid x \notin A\}$$

Algorithm:

1. Start.
2. Define the universe of discourse U.
3. Define crisp sets A and B as subsets of U.
4. Perform:
   - Union → combine elements of both sets.
   - Intersection → common elements of both sets.
   - Difference → elements of one set not in the other.
   - Complement → elements in U but not in the set.

5. Display the results.
6. Stop.

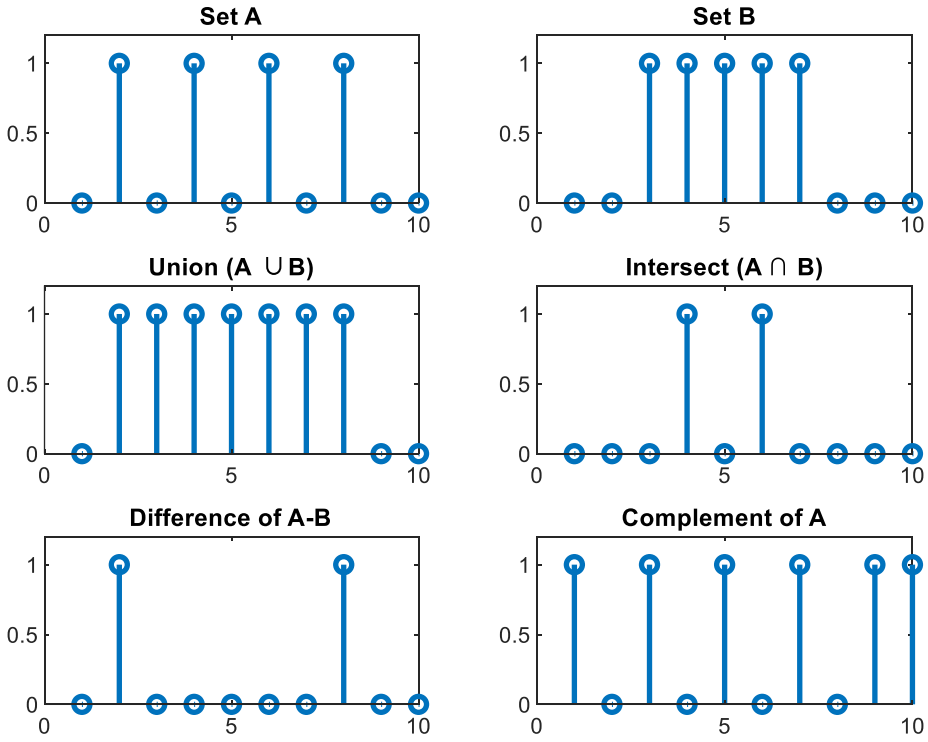
Program:

```
clc;
clear;
% Step 1: Define universe
U = 1:10;
% Step 2: Define two crisp sets
A = [2 4 6 8];
B = [3 4 5 6 7];
% Step 3: Operations
unionAB = union(A,B);
intersectAB = intersect(A,B);
diffAB = setdiff(A,B);      % A - B
compA = setdiff(U,A);       % Complement of A
% Step 4: Display results
disp(['Universe U = ', num2str(U)]);
disp(['Set A = ', num2str(A)]);
disp(['Set B = ', num2str(B)]);
disp(['Union (A ∪ B) = ', num2str(unionAB)]);
disp(['Intersection (A ∩ B) = ', num2str(intersectAB)]);
disp(['Difference (A - B) = ', num2str(diffAB)]);
disp(['Complement of A = ', num2str(compA)]);
% Step 5: Plot Venn-like visualization
figure;
subplot(3,2,1); stem(U, ismember(U,A),'LineWidth',2); title('Set A'); ylim([0 1.2]);
subplot(3,2,2); stem(U, ismember(U,B),'LineWidth',2); title('Set B'); ylim([0 1.2]);
subplot(3,2,3); stem(U, ismember(U,unionAB),'LineWidth',2); title('Union (A ∪ B)');
ylim([0 1.2]);
subplot(3,2,4); stem(U, ismember(U,intersectAB),'LineWidth',2); title('Intersect (A ∩ B)');
ylim([0 1.2]);
subplot(3,2,5); stem(U, ismember(U,diffAB),'LineWidth',2); title('Difference of A-B');
```

```
ylim([0 1.2]);
subplot(3,2,6); stem(U, ismember(U,compA),'LineWidth',2); title('Complement of A');
ylim([0 1.2]);
```

Output:

```
Universe U = 1   2   3   4   5   6   7   8   9  10
Set A = 2  4  6  8
Set B = 3  4  5  6  7
Union (A ∪ B) = 2  3  4  5  6  7  8
Intersection (A ∩ B) = 4  6
Difference (A - B) = 2  8
Complement of A = 1   3   5   7   9  10
```



Results:

The operations of crisp sets were successfully implemented in MATLAB. The union combined elements from both sets, the intersection gave common elements, the difference gave elements present in one set but not the other, and the complement gave elements from the universe not present in the set. Thus, crisp set operations were verified successfully.

Ex.No: 09 **IMPLEMENTATION OF LOGIC GATES** **Date:**

Aim:

To implement basic logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR) using MATLAB and verify their truth tables.

Theory:

Logic gates are the fundamental building blocks of digital systems. Each gate performs a logical operation on one or more binary inputs to produce a single binary output.

Basic Logic Gate Operations:

NOT Gate: (Inverts input)

$$Y = \bar{A}$$

AND Gate: (Only 1 if both inputs are 1)

$$Y = A \cdot B$$

OR Gate: (1 if either input is 1)

$$Y = A + B$$

NAND Gate: (Only 0 if both inputs are 1)

$$Y = \overline{A \cdot B}$$

NOR Gate: (0 if either input is 1)

$$Y = \overline{A + B}$$

XOR Gate (Exclusive OR): (Outputs **1** only when the two inputs are **different)**

$$Y = A \oplus B = (A \cdot \bar{B}) + (\bar{A} \cdot B)$$

XNOR Gate (Exclusive NOR): (Outputs **1** only when the two inputs are **same)**

$$Y = \overline{A \oplus B}$$

Algorithm:

1. Start.
2. Define all possible binary inputs $A, B \in \{0,1\}$.
3. Apply logical operators $(\&, |, \sim, xor)$ in MATLAB.
4. Generate truth tables for each gate.
5. Display results.

6. Stop.

Program:

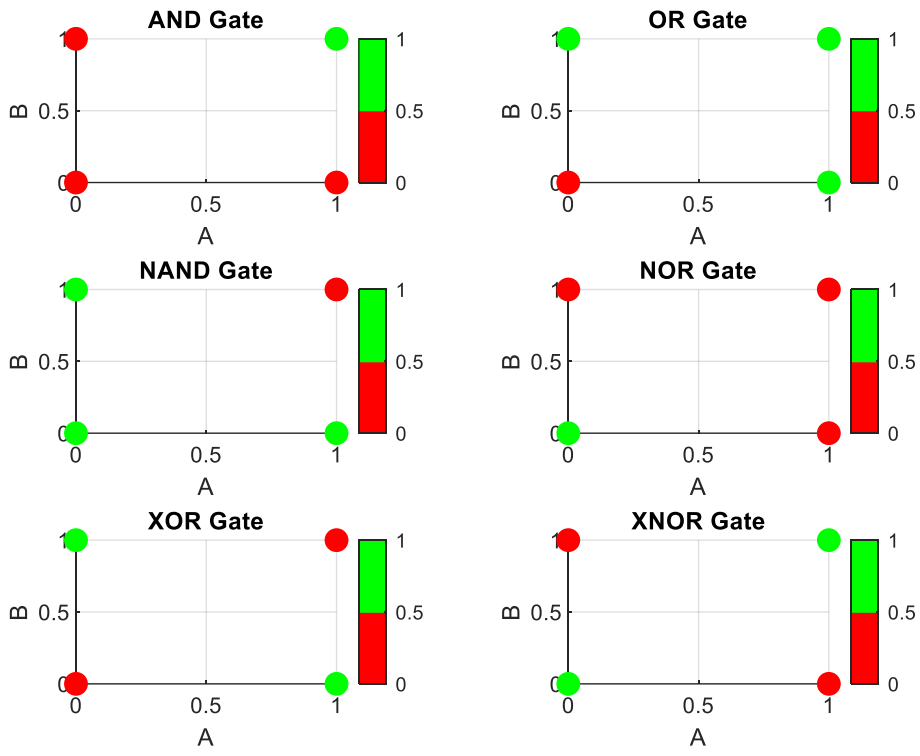
```
clc; clear;
% Step 1: Define all possible inputs
A = [0 0 1 1]';
B = [0 1 0 1]';
% Step 2: Implement logic gates
NOTA = ~A;
NOTB = ~B;
AND  = A & B;
OR   = A | B;
NAND = ~(A & B);
NOR  = ~(A | B);
XOR = xor(A, B);
XNOR = ~xor(A, B);
% Step 3: Display truth tables
T = table(A, B, NOTA,NOTB,AND,OR,NAND, NOR, XOR, XNOR);
disp('Truth Table for Logic Gates:');
disp(T);
% Step 4: Plot input combinations (A,B) and outputs
figure;
subplot(3,2,1);
scatter(A,B,80,AND,'filled');
title('AND Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
colorbar;
subplot(3,2,2);
scatter(A,B,80,OR,'filled');
title('OR Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
```

```
colorbar;
subplot(3,2,3);
scatter(A,B,80,NAND,'filled');
title('NAND Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
colorbar;
subplot(3,2,4);
scatter(A,B,80,NOR,'filled');
title('NOR Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
colorbar;
subplot(3,2,5);
scatter(A,B,80,XOR,'filled');
title('XOR Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
colorbar;
subplot(3,2,6);
scatter(A,B,80,XNOR,'filled');
title('XNOR Gate');
xlabel('A'); ylabel('B'); grid on;
colormap([1 0 0; 0 1 0]); % red=0, green=1
colorbar;
```

Output:

```
Truth Table for Logic Gates:
    A     B     NOTA     NOTB     AND      OR      NAND     NOR      XOR      XNOR
    _     _    _____    _____    _____    _____    _____    _____    _____    _____

    0     0    true     true     false    false    true     true     false    true
    0     1    true     false    false    true     true     false    true     false
    1     0    false    true     false    true     true     false    true     false
    1     1    false    false    true     true     false    false    false    true
```



Result:

The basic logic gates AND, OR, NOT, NAND, NOR, XOR, and XNOR were successfully implemented in MATLAB. Their truth tables were generated and matched with the expected outputs. The behavior of the logic gates was verified and visualized using plots.

Ex.No: 10 IMPLEMENTATION OF GENETIC ALGORITHMS Date:

Aim:

To study and implement the **Genetic Algorithm (GA)** for solving optimization problems and obtain the optimal solution with MATLAB.

Algorithm:

Initialize Population:

- Generate a random initial population of N individuals (chromosomes).
- Each chromosome represents a candidate solution.

Chromosome Encoding:

- Represent variables as binary strings or real values.
- Example: For binary GA,

$$x = x_{min} + \frac{decimal(chromosome)}{2^L - 1}(x_{max} - x_{min})$$

$where\ L\ =\ chromosome\ length.$

Fitness Evaluation:

- Evaluate the **fitness** of each chromosome using the objective function $f(x)$.
- For minimization:

$$Fitness = \frac{1}{1 + f(x)}$$

Selection:

- Select parents according to fitness values.
- Methods: **Roulette Wheel**, **Tournament**, **Rank Selection**.

Crossover (Recombination):

- With probability Pc, choose a crossover point and exchange chromosome parts between parents.
- Example (single-point crossover):

  Parent1: 10101

  Parent2: 11011

  Crossover → Child1: 10111, Child2: 11001

Mutation:

- With probability Pm, flip bits or add perturbation.

- Ensures diversity and avoids premature convergence.

New Generation Formation:

- Combine parents and offspring to form the next generation.

Termination:

- Repeat until **maximum generations** reached or solution **converges**.

Mathematical Formulas in GA

Chromosome Decoding

$$x = x_{min} + \frac{decimal(binary)}{2^L - 1}(x_{max} - x_{min})$$

Fitness Function

$$Fitness = \frac{1}{1 + f(x)} \quad \text{(for minimization)}$$

Selection Probability

$$P_i = \frac{Fitness_i}{\sum_{j=1}^{N} Fitness_j}$$

Crossover

$$Child = [Parent1(1:k), Parent2(k+1:L)]$$

Mutation

$$bit' = \neg(bit)$$

Procedure

1. Define the optimization problem (objective function, constraints, bounds).
2. Initialize GA parameters:
    - Population size (N)
    - Chromosome length (L)
    - Crossover probability (Pc)
    - Mutation probability (Pm)
    - Maximum generations
3. Generate the initial population randomly.
4. Decode chromosomes into problem variables.
5. Evaluate the fitness of each chromosome.

6. Apply selection, crossover, and mutation operators.
7. Form the next generation and repeat evaluation.
8. Continue until termination criteria are met.
9. Display the optimal solution and fitness value.

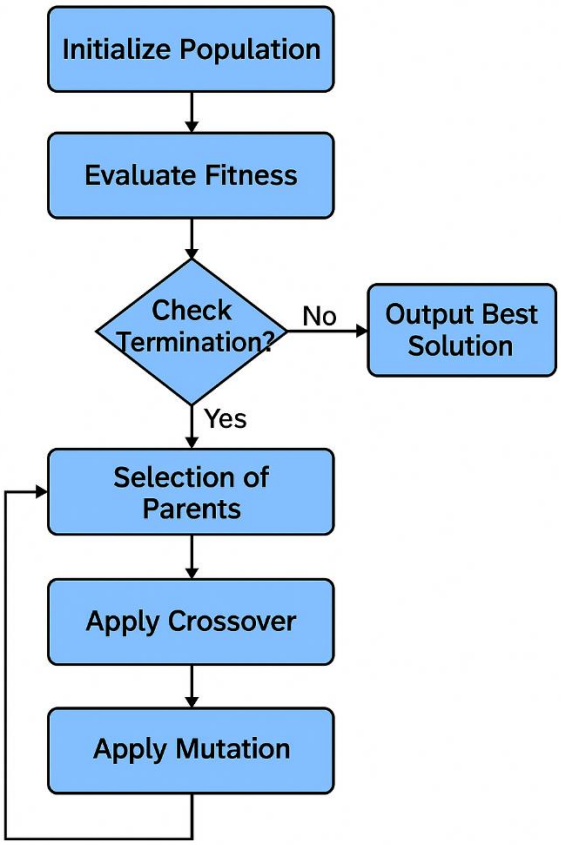
Matlab Command for GA

[x,fval] = ga(fitnessFunc, nvars, [], [], [], [], lb, ub, [], options);

GA Options

Option	Meaning
'PopulationSize'	Number of individuals in each generation
'MaxGenerations'	Maximum number of generations
'CrossoverFraction'	Proportion of crossover vs mutation
'MutationFcn'	Mutation function
'Display'	'off', 'iter', 'final'
'PlotFcn'	Plot functions like @gaplotbestf (best fitness)
'EliteCount'	Number of elite individuals preserved
'FunctionTolerance'	Stop if improvement < tolerance

Flow Chart



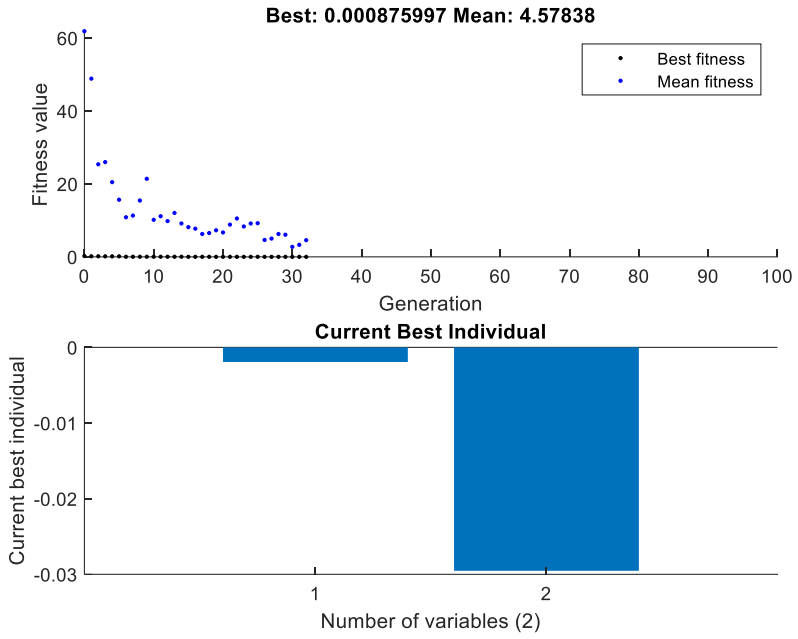
Program:

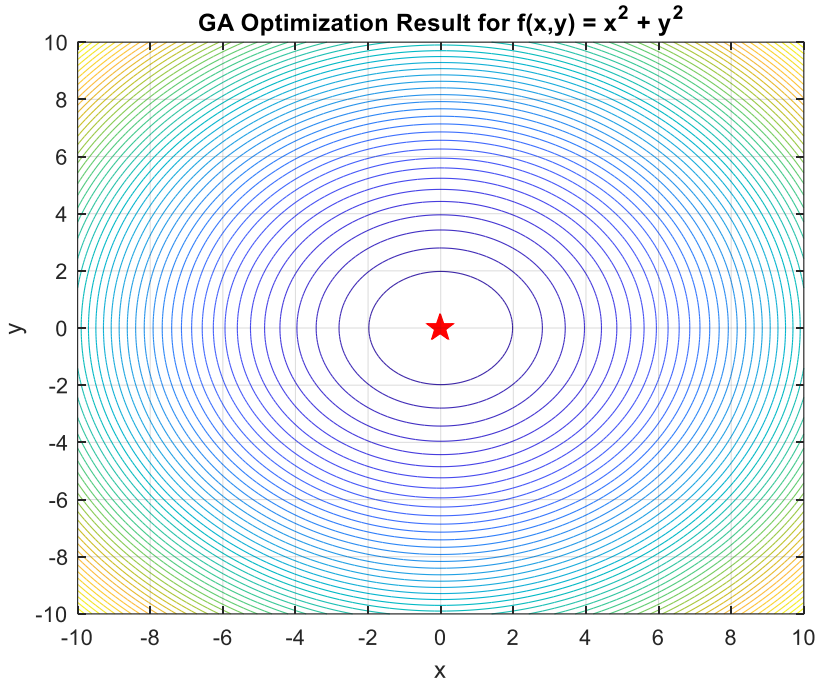
```
clc; clear; close all;
% Objective function
fitnessFunc = @(v) v(1).^2 + v(2).^2;   % f(x,y) = x^2 + y^2
% Number of variables
nvars = 2;
% Lower and upper bounds
lb = [-10 -10];
ub = [10 10];
% GA Options
options = optimoptions('ga', ...
   'Display','iter', ...                % show iteration details
   'MaxGenerations',100, ...            % maximum number of generations
   'PopulationSize',50, ...             % number of individuals
   'CrossoverFraction',0.8, ...         % fraction of crossover
   'MutationFcn',{@mutationgaussian,0.2}, ... % Gaussian mutation
   'OutputFcn',@myGAcontrol, ...        % Custom output function
   'PlotFcn',{@gaplotbestf,@gaplotbestindiv}); % Plots
% Run GA
[x,fval] = ga(fitnessFunc, nvars, [], [], [], [], lb, ub, [], options);
% Display results
disp(['Optimal solution: x = ', num2str(x(1)), ', y = ', num2str(x(2))]);
disp(['Minimum value: f(x,y) = ', num2str(fval)]);
% Plot contour and optimal point
[X,Y] = meshgrid(-10:0.1:10, -10:0.1:10);
Z = X.^2 + Y.^2;
figure;
contour(X,Y,Z,50); hold on;
%plot(x(1),x(2),'rp','MarkerSize',12,'MarkerFaceColor','r');
plot(x,fval,'rp','MarkerSize',12,'MarkerFaceColor','r');
title('GA Optimization Result for f(x,y) = x^2 + y^2');
xlabel('x'); ylabel('y'); grid on;
```

```
%% ======================
% Custom Output Function
% ======================
function [state,options,optchanged] = myGAcontrol(options,state,flag)
   optchanged = false;
   switch flag
      case 'iter'
         % Show population of this generation
         fprintf('Generation %d Population:\n', state.Generation);
         disp(state.Population);
         % Early stopping if best fitness below threshold
         bestFitness = min(state.Score);
         if bestFitness <= 1e-3
            fprintf('Stopping early: best fitness = %.6f\n', bestFitness);
            state.StopFlag = 'y';
         end
   end
end
```

Output:

```
Single objective optimization:2 Variable(s)
Options:
CreationFcn:       @gacreationuniform
CrossoverFcn:      @crossoverscattered
SelectionFcn:      @selectionstochunif
MutationFcn:       @mutationgaussian
Stopping early: best fitness = 0.000876
                          Best          Mean     Stall
Generation     Func-count     f(x)          f(x)    Generations
     32          1557          0.000876     4.578       0
Optimization terminated: y
Optimal solution: x = -0.0019322, y = -0.029534
Minimum value: f(x,y) = 0.000876
```





Results:

The Genetic Algorithm was applied to minimize the function $f(x, y) = x^2 + y^2$ within the range $[-10, 10]$ for both variables. The algorithm started with a random initial population and evolved solutions using selection, crossover, and Gaussian mutation. During the iterations, the best fitness value improved steadily and the algorithm successfully converged to the global minimum. The optimal solution obtained was approximately **x = -0.0019322 and y = -0.029534**, with the corresponding minimum function value **f(x,y) = 0.000876**. The contour plot confirmed that the GA correctly located the minimum point at the origin.

Ex.No: 11 IMPLEMENTATION OF CLASSIFICATION ALGORITHM Date:

Aim:

To implement and compare different classification algorithms — **Support Vector Machine (SVM), k-Nearest Neighbors (kNN), Decision Tree, Random Forest, and Neural Network** — using the Fisher's Iris dataset in MATLAB, and to evaluate their performance using confusion matrices and accuracy.

Algorithm:

1. **Load Dataset**
   - Import the Fisher's Iris dataset (load fisheriris).
   - Extract features (meas) and labels (species).
2. **Data Preprocessing**
   - Split the dataset into training (70%) and testing (30%) using cvpartition.
3. **Model Training**

   Train classifiers:
   - SVM using fitcecoc.
   - kNN using fitcknn.
   - Decision Tree using fitctree.
   - Random Forest using fitcensemble.
   - Neural Network using fitnet (with one-hot encoding).
4. **Prediction**
   - Use each model to predict the class labels of the test dataset.
5. **Evaluation**
   - Compute the accuracy = (Correct Predictions ÷ Total Test Samples) × 100.
   - Generate and analyze confusion matrices using confusionchart.

Procedure:

1. Start MATLAB and create a new script.
2. Load the Fisher's Iris dataset (load fisheriris).
3. Partition the dataset into training and testing sets.
4. Train five classifiers:
   - **SVM (fitcecoc) for multi-class classification.**
   - **kNN (fitcknn) with 5 neighbors.**
   - **Decision Tree (fitctree) for rule-based classification.**

- **Random Forest (fitcensemble) using bagged trees.**
- **Neural Network (fitnet) with 10 hidden neurons.**

5. Predict the labels for test data using each trained model.
6. Plot confusion matrices to visualize classification performance.
7. Compute accuracy values for each classifier.
8. Compare the results to identify the best classifier.

Classification & Regression

- **fitcsvm** → Fit a **binary SVM classifier**.
- **fitcecoc** → Fit a **multiclass classifier** using Error-Correcting Output Codes (ECOC) with SVM or other learners.
- **fitcknn** → Fit a **k-nearest neighbor classifier**.
- **fitctree** → Fit a **classification decision tree**.
- **fitcensemble** → Fit an **ensemble classifier** (e.g., bagged trees, boosting, random forest).
- **fitclinear** → Fit a linear model for classification.
- **fitcnb** → Fit a **Naive Bayes classifier**.
- **fitcrm** → Fit a classification model for **cost-sensitive learning** (less common).
- **fitrsvm** → Fit a **Support Vector Machine regression model**.
- **fitrlinear** → Fit a **linear regression model**.
- **fitrtree** → Fit a **regression decision tree**.
- **fitrensemble** → Fit an **ensemble regression model**.
- **fitrgp** → Fit a **Gaussian process regression model**.
- **fitrkernel** → Fit a **kernel regression model**.
- **fitrm** → Fit a **repeated measures model**.

Neural Networks (Deep Learning Toolbox)

- **fitnet** → Create and train a **neural network for function fitting** (regression).

Curve Fitting & Statistics

- **fit** → Fit **curves or surfaces** to data (from Curve Fitting Toolbox).
- **fitdist** → Fit a **probability distribution** object to data.
- **fitlm** → Fit a **linear regression model**.
- **fitglm** → Fit a **generalized linear model**.
- **fitnlm** → Fit a **nonlinear regression model**.
- **fitglme** → Fit a **generalized linear mixed-effects model**.

- **fitlme** → Fit a **linear mixed-effects model**.

Program:

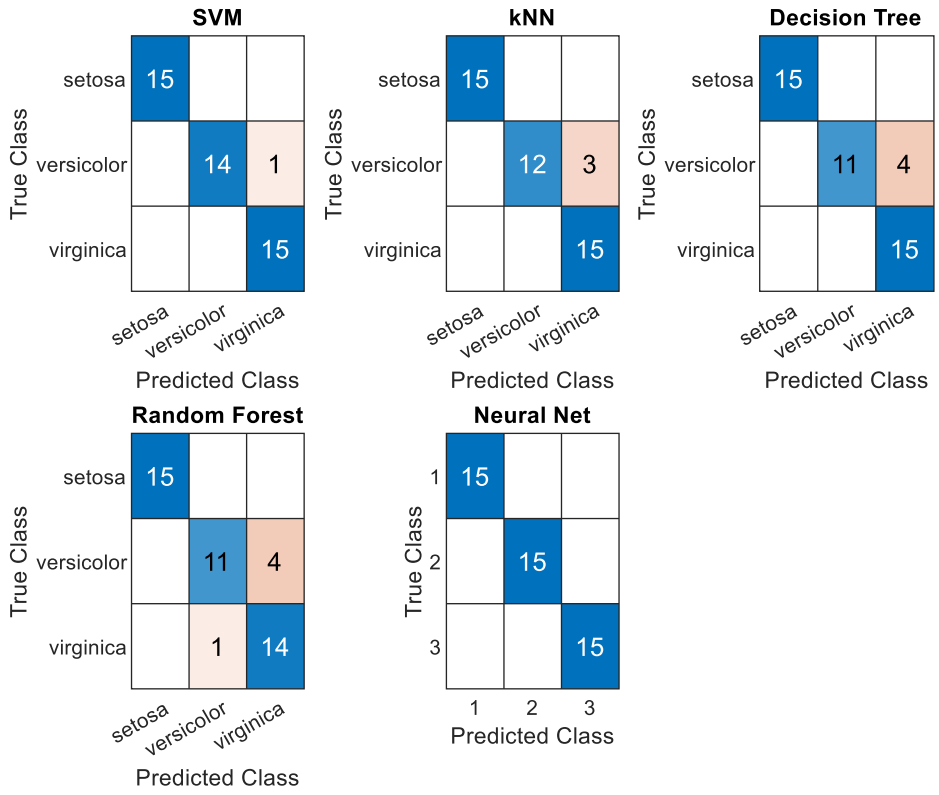
```
clc; clear; close all;
%% Load Dataset
load fisheriris
X = meas;        % Features (150x4)
Y = species;     % Labels (150x1 cell)
%% Split dataset (70% train, 30% test)
cv = cvpartition(Y,'HoldOut',0.3);
Xtrain = X(training(cv),:);
Ytrain = Y(training(cv));
Xtest  = X(test(cv),:);
Ytest  = Y(test(cv));
%% 1. Support Vector Machine (SVM)
svmModel = fitcecoc(Xtrain,Ytrain);
Ypred_svm = predict(svmModel,Xtest);
%% 2. k-Nearest Neighbors (kNN)
knnModel = fitcknn(Xtrain,Ytrain,'NumNeighbors',5);
Ypred_knn = predict(knnModel,Xtest);
%% 3. Decision Tree
treeModel = fitctree(Xtrain,Ytrain);
Ypred_tree = predict(treeModel,Xtest);
%% 4. Random Forest (Ensemble of Decision Trees)
rfModel = fitcensemble(Xtrain,Ytrain,'Method','Bag');
Ypred_rf = predict(rfModel,Xtest);
%% 5. Neural Network using fitnet
X_nn = X';              % transpose for neural net (features x samples)
Ynum = grp2idx(Y);      % convert species to numeric (1,2,3)
Y_onehot = full(ind2vec(Ynum')); % one-hot encoding
% Split for train/val/test
net = fitnet(10);
net.divideParam.trainRatio = 0.7;
```

```
net.divideParam.valRatio   = 0.15;
net.divideParam.testRatio  = 0.15;
[net,~] = train(net,X_nn,Y_onehot);   % train NN
Ypred_nn = net(Xtest');               % test NN
[~,Ypred_nn_class] = max(Ypred_nn);   % get class labels
Ypred_nn = categorical(Ypred_nn_class)';
Ytest_nn = categorical(grp2idx(Ytest));
%% Confusion Matrices
figure;
subplot(2,3,1); confusionchart(Ytest,Ypred_svm); title('SVM');
subplot(2,3,2); confusionchart(Ytest,Ypred_knn); title('kNN');
subplot(2,3,3); confusionchart(Ytest,Ypred_tree); title('Decision Tree');
subplot(2,3,4); confusionchart(Ytest,Ypred_rf); title('Random Forest');
subplot(2,3,5); confusionchart(Ytest_nn,Ypred_nn); title('Neural Net');
%% Accuracy Results
acc_svm  = sum(strcmp(Ypred_svm,Ytest))/length(Ytest)*100;
acc_knn  = sum(strcmp(Ypred_knn,Ytest))/length(Ytest)*100;
acc_tree = sum(strcmp(Ypred_tree,Ytest))/length(Ytest)*100;
acc_rf   = sum(strcmp(Ypred_rf,Ytest))/length(Ytest)*100;
acc_nn   = sum(Ypred_nn == Ytest_nn)/length(Ytest)*100;
fprintf('Accuracy (SVM): %.2f%%\n',acc_svm);
fprintf('Accuracy (kNN): %.2f%%\n',acc_knn);
fprintf('Accuracy (Decision Tree): %.2f%%\n',acc_tree);
fprintf('Accuracy (Random Forest): %.2f%%\n',acc_rf);
fprintf('Accuracy (Neural Network): %.2f%%\n',acc_nn);
```

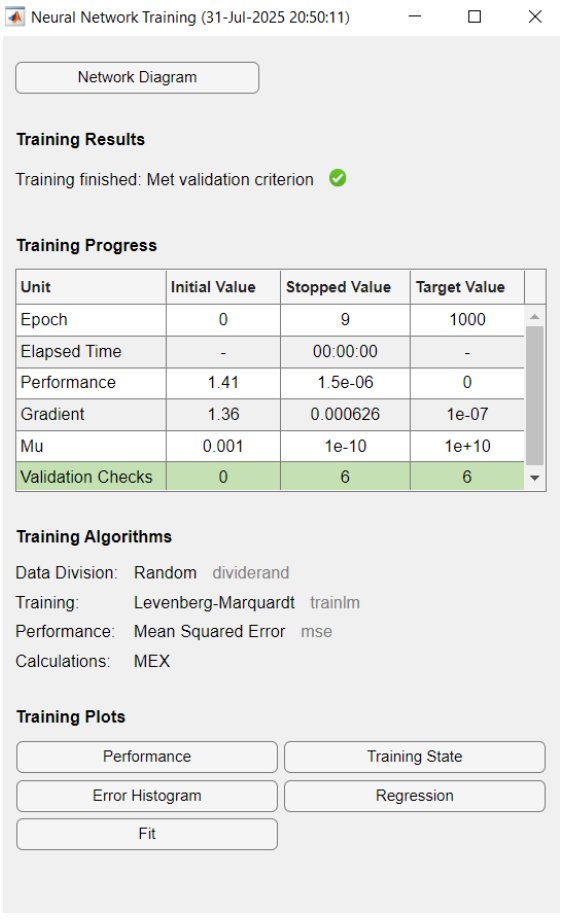
Output:

```
Accuracy (SVM): 97.78%
Accuracy (kNN): 93.33%
Accuracy (Decision Tree): 91.11%
Accuracy (Random Forest): 88.89%
Accuracy (Neural Network): 100.00%
```

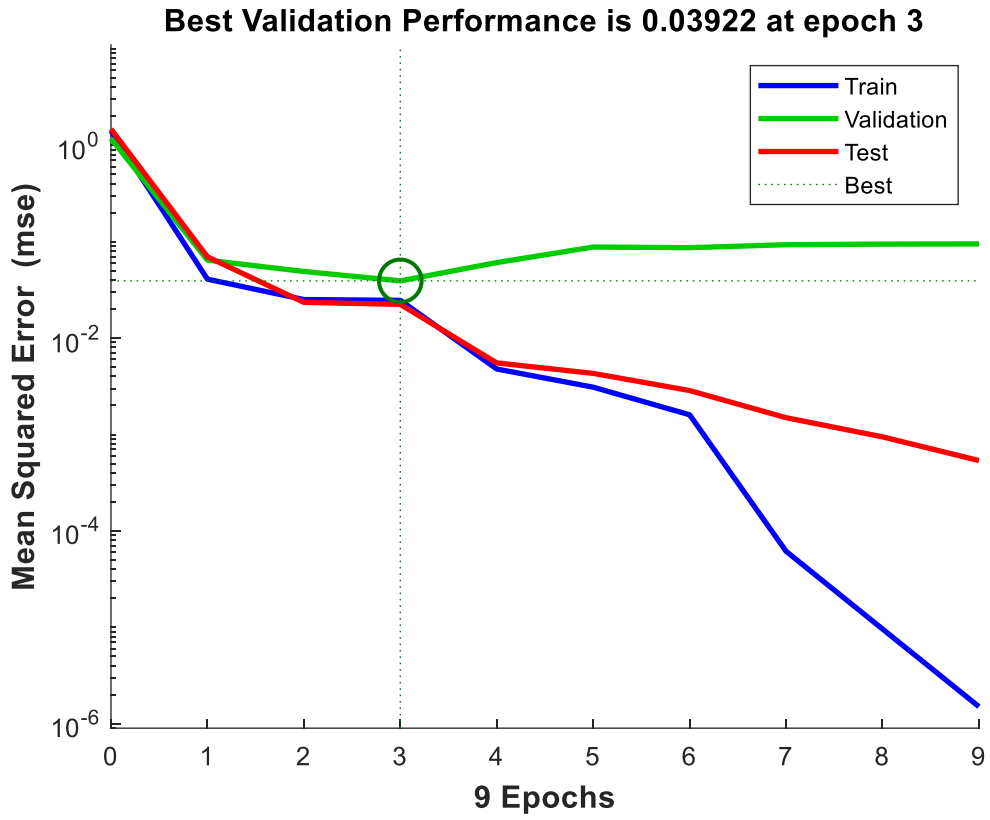
Confusion Matrices:



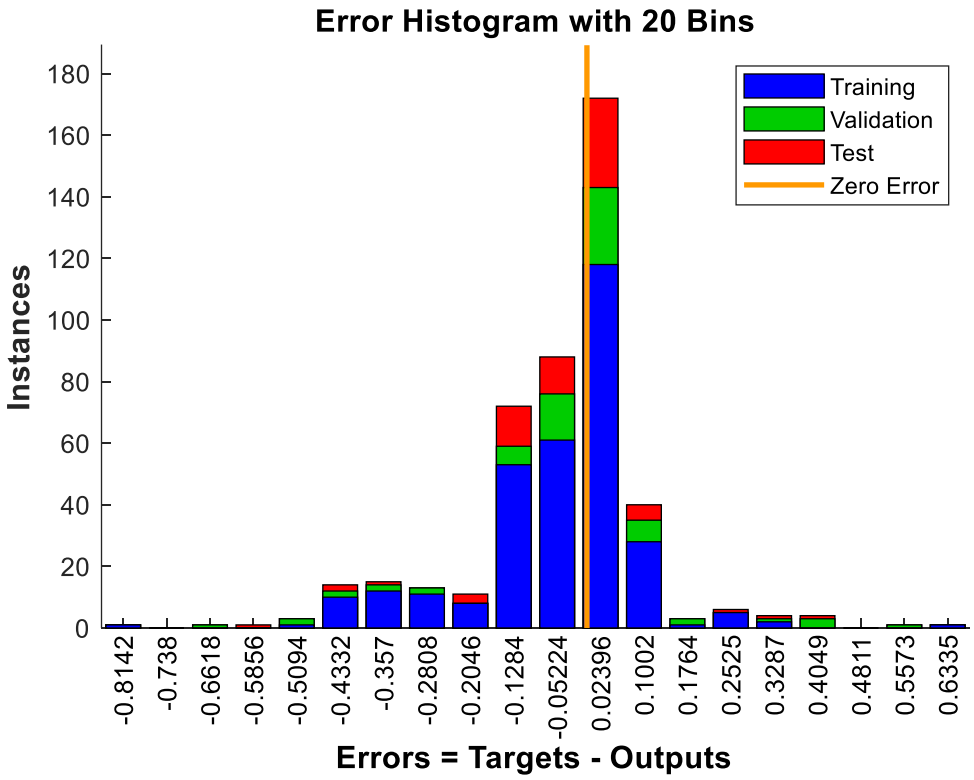
Network Diagram:



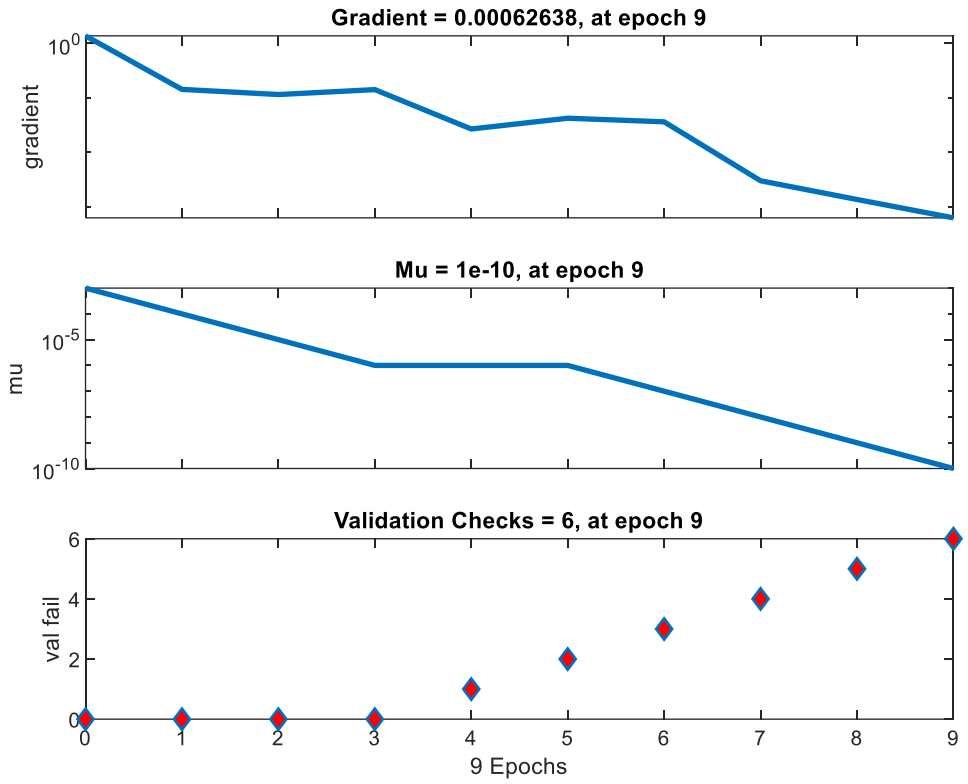
Training Performance:



Neural Network Training Error:



Training State:



Results:

The classification algorithms were implemented on the Fisher's Iris dataset in MATLAB and their performances were evaluated. The **Support Vector Machine (SVM)** achieved an accuracy of **97.78%**, while the **k-Nearest Neighbors (kNN)** classifier obtained an accuracy of **93.33%**. The **Decision Tree** classifier gave a lower accuracy of **91.11%**, and the **Random Forest** model achieved **88.89%**. The **Neural Network (fitnet)** outperformed all other models with a perfect accuracy of **100.00%**. From these results, it can be concluded that the Neural Network provided the best classification performance, followed by SVM, while the Random Forest had the lowest accuracy among the tested classifiers.